

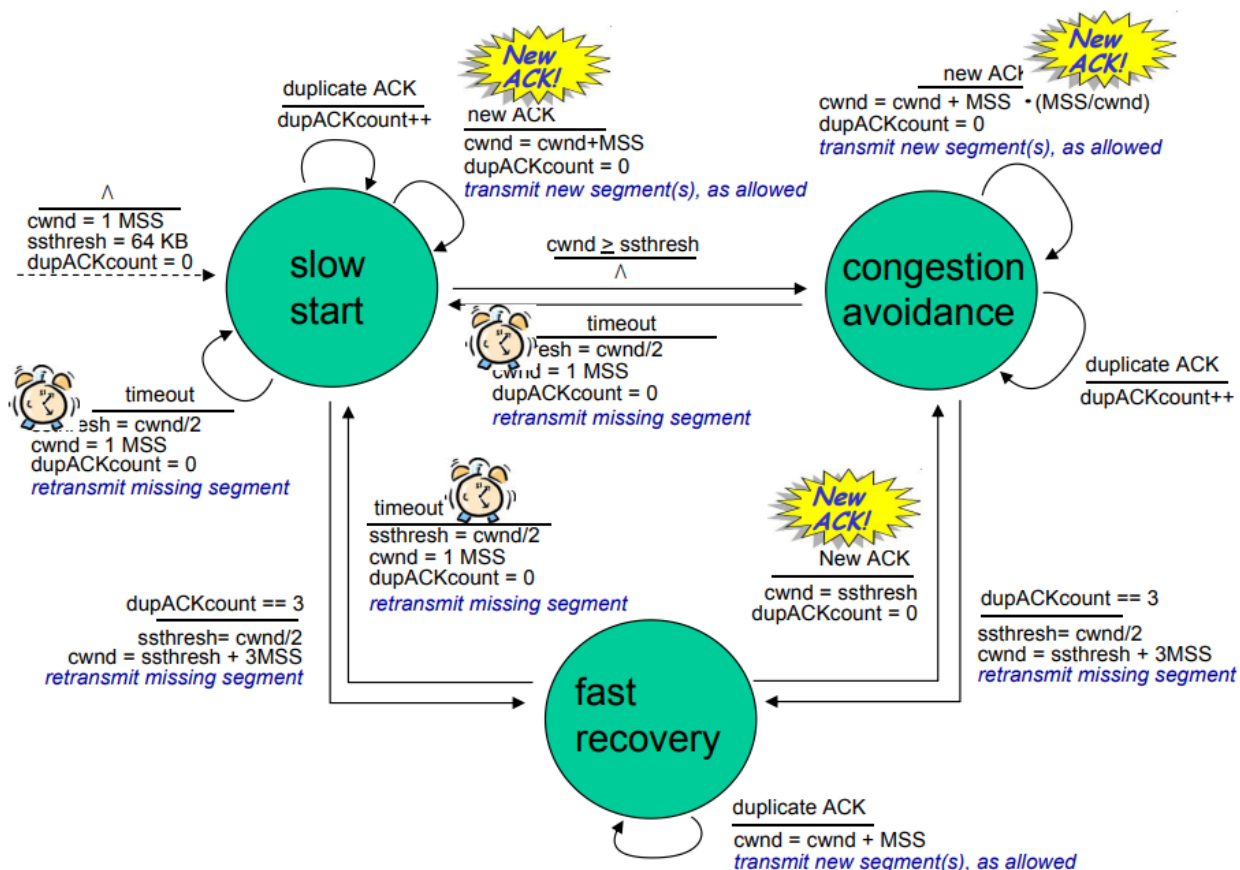
lecture 9 (Transport Layer -- congestion control)

• principles of congestion control

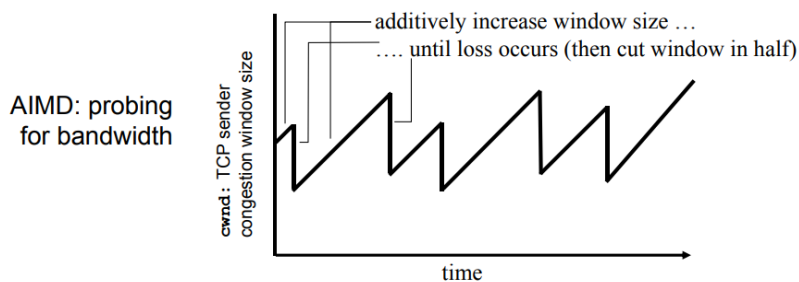
- Congestion 拥塞：
- Causes/costs of congestion: scenario 1
- Causes/costs of congestion: scenario 2
- Causes/costs of congestion: scenario 3
- Cause and Cost of Congestion：
 - Cause 原因：
 - 共享链路；链路容量有限 —— 多流竞争同一瓶颈链路导致拥塞
 - 以高速率发送 —— 发送速率过高会超出网络处理能力
 - Cost 代价：
 - 延迟 —— 排队带来时延增加
 - 丢包与重传
 - 不必要的重传：浪费
 - 上游传输容量被浪费
- Approaches to Congestion Control：
 - 端到端拥塞控制 —— 仅靠通信双方（不用路由器协助）的控制方法：
 - TCP 通过监测丢包或 RTT 增长来察觉拥塞
 - 检测到拥塞信号（丢包/延迟）后，TCP 缩小发送窗口以降速缓解拥塞。
 - 网络辅助的拥塞控制 —— 路由器/网络节点参与提供反馈或控制的方案：
 - 路由器向发送方和/或接收方提供反馈
 - 网络可直接告知端点当前拥塞状态（显式信号）
 - 用1bit指示链路上的拥塞；或告知路由器可支持的主机最大发送速率
 - 示例机制包括显式拥塞标记（ECN）或路由器指示可接受的发送速率。

• TCP congestion control

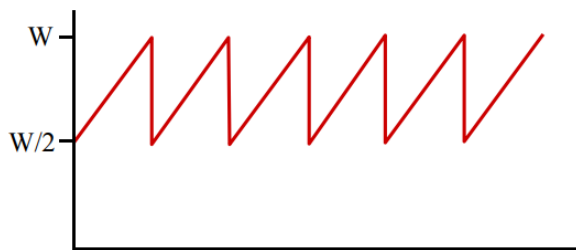
- TCP 常用策略是逐步增加发送窗口（线性增），遇拥塞时按比例减少（乘性减）。
 - TCP 使用端到端的拥塞控制。
 - 端点（发送方）通过观测端到端信号来决定速率，而不是依赖路由器直接控制。
 - 让每个发送方根据其感知到的网络拥塞来限制发送速率。
 - 发送方根据丢包/延迟等信号调整发送速率，避免过度占用网络资源。
- 实现拥塞控制需要回答的问题：
- 拥塞控制算法有三个主要组成部分：



- TCP 拥塞控制：加性增加，乘性减少（AIMD）。
 - 方法：发送方增加发送速率（即窗口大小），探测可用带宽，直到发生丢包为止。
 - 加性增加：在每个 RTT 中将 $cwnd$ 增加 1 个 MSS，直到检测到丢包。
 - 线性、温和地增加窗口以避免突发拥塞。
 - 乘性减少：在发生丢包后将 $cwnd$ 减半。
 - 遇到拥塞时按比例快速降速以减轻网络负担。
 - AIMD：探测带宽（曲线锯齿形，上升——发生丢包——回退）



- TCP 吞吐量



- 平均 TCP 吞吐量如何随窗口大小和 RTT 变化？
 - 要估算长期平均速率需考虑窗口与 RTT。
 - 忽略慢启动，假设总有数据可发送。
- W：发生丢包时的窗口大小（以字节计）。
- 平均窗口大小（在途字节数）为 $3/4 W$ 。
 - 锯齿从 $W/2$ 跳到 W ，均值约为 $0.75W$ （标准简化结果）。
- 平均吞吐约为 $(3/4 \cdot W) / \text{RTT}$ （字节/秒）
 - 吞吐 = 平均在途字节 / RTT。

- TCP Futures: TCP over “long, fat pipes”:

TCP Futures: TCP over “long, fat pipes”

$$\text{avg TCP throughput} = \frac{3}{4} \frac{W}{\text{RTT}} \text{ bytes/sec}$$

- ❖ example: 1500 byte ^{MSS} segments, 100ms ^{0.1s} RTT, want 10 Gbps throughput $\rightarrow 10 \times 10^9 \text{ bytes/s} = 10^{10} \text{ bytes/s} = 1.25 \times 10^8 \text{ bytes/s}$
- ❖ requires $W = 83,333$ in-flight segments $= \frac{1.25 \times 10^8 \times 0.1}{1500}$
- ❖ throughput in terms of segment loss probability, L [Mathis 1997]:

$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT} \sqrt{L}}$$

$\rightarrow$ to achieve 10 Gbps throughput, need a loss rate of $L = 2 \cdot 10^{-10}$ – a very small loss rate!

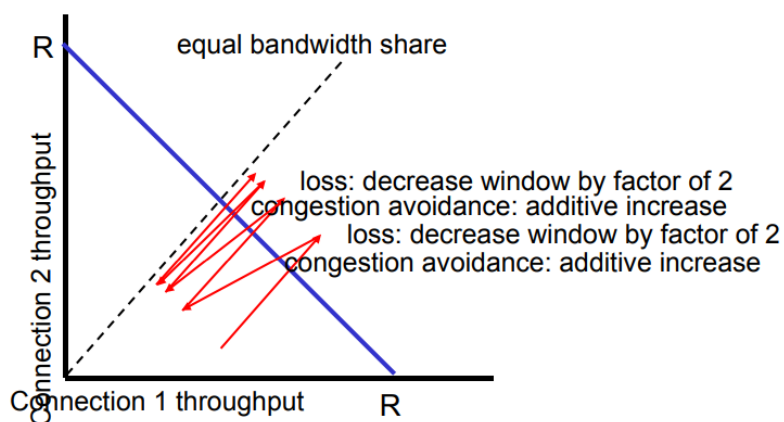
- ❖ new versions of TCP for high-speed

- TCP 吞吐 $\approx 1.22 \times \text{MSS} / (\text{RTT} \times \sqrt{L})$
- 此公式给出在丢包概率 L 下的平均吞吐估算（适用于 AIMD 模型）
- 在长延迟高带宽路径上，要靠传统 TCP 达到高速，需要几乎为零的丢包率，这在现实中难以实现。
- 为高带宽场景提出了新的 TCP 版本（高速 TCP）。
 - 为了解决“长胖管道”问题，研究并设计了适合高带宽大 RTT 的改进算法（如 CUBIC、BBR 等）。

- TCP 公平性

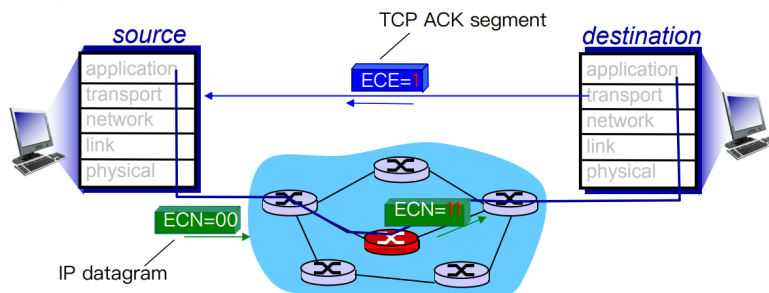
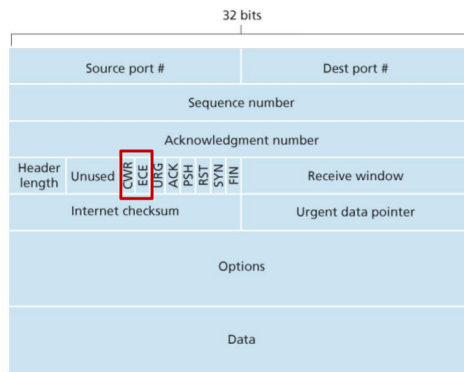
- 目标：若 K 个 TCP 会话共享同一瓶颈链路（带宽 R），每个会话的平均速率应为 R/K 。

- 分析两条流共享同一瓶颈的情况：



- 加性增加在吞吐增加时给出斜率为 1 的增长（方向）
 - 两个连接在拥塞避免中每 RTT 各自线性地增加同样量，沿对角线方向移动（相对增加相同）。
- 乘性减少按比例降低吞吐
 - 发生丢包时窗口按比例缩小（例如减半），保持相对比例不变。
- 图注（虚线）：带宽平分线（等带宽份额）
 - 表示两个连接获得相等带宽的点。
- 出现丢包→窗口减半（向内跳）；随后加性增加→向外缓慢爬升；反复后趋向对角线附近。
- 解释：AIMD 的“减半—线性增”使两条流在长期内趋向平分带宽，因此被认为有“公平性”特性。
- UDP 对 TCP 公平性的影响。
 - 多媒体应用常常不使用 TCP —— 实时性/延迟敏感
 - 于是使用 UDP，UDP 无重传/拥塞控制，便于恒定发送
 - 以恒定速率发送音/视频，容忍丢包。
 - 代价是会在拥塞时“抢占”带宽。UDP 不服从拥塞控制会挤占 TCP。
 - UDP 源会把 TCP 流挤出（占用带宽）。
 - UDP 不退让，会导致对 TCP 不公平。
- 另一个破坏公平性的实践——并行 TCP 连接
 - 应用可以在两台主机间打开多个并行连接，比如同时开启若干 TCP 流
 - 浏览器并行下载多个资源，利用多个 TCP 会话
 - 例如：链路带宽为 R ，已有 9 个连接：
 - 新应用请求 1 个 TCP，会得到 $R/10$ 的速率。
 - 若新应用同时打开 11 个 TCP，会得到超过 $R/2$ 的速率。
 - 通过开很多并行连接，一个应用能“赢得”超过公平份额（不公平）。
- 显式拥塞通知（ECN）

- 网络辅助的拥塞反馈机制
- 由路由器直接参与通知端点拥塞状况
- IP 报头（服务类型字段）中的2 bit由网络路由器标记以指示拥塞。
 - 路由器在缓冲趋满时可打标记（不丢包）以示拥塞。
- 拥塞指示被传递到接收端主机，接收端在收到带标记的 IP 包后知道链路有拥塞。
- 接收端在回送给发送端的 ACK 段上置位 ECE（拥塞已体验）位，以通知发送端发生拥塞。



- IP 数据包起初 ECN=00（未使用）；路由器检测到拥塞将 ECN 标记为 ECN=11（或 CE）；到达接收端后，接收端在 ACK 中设置 ECE=1 通知发送端。
- ECN 意味着路由器可以“标记而非丢弃”，让端点更早响应拥塞以减少丢包和重传。